

AWS STATIC WEBSITE HOSTING WITH CI/CD FOR MODERN CLOUD DEPLOYMENT

*Prepared in the partial fulfillment of the Summer Internship Program on AWS
AT*



Under the guidance of

Under the guidance of

Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. J. Lovababu, APSSDC

Submitted by

SAI SANDEEP GEDELA, 323506402147, Andhra University College of Engineering, (Batch -2)

TEDLAPU HEMAVARDHAN RAM SAI, 324506402615, Andhra University College of Engineering, (Batch -1)

TEDLAPU HIMA VARDHAN LAXMAN SAI, 324506402616, Andhra University College of Engineering, (Batch -1)

PENUGONDA MANASA LAKSHMI, Y24CS266, R.V.R and J.C College of Engineering, (Batch -1)

REVTISH MUTHINENI, Y24CS294, R.V.R and J.C College of Engineering, (Batch -1)

Internship Duration: 15/05/2026 to 14/07/2026

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at Andhra Pradesh Skill Development Corporation (APSSDC). This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr J. Lovababu, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at Andhra Pradesh Skill Development Corporation (APSSDC) for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Sai Sandeep Gedela, 323506402147, saisandeepgedela18@gmail.com.

Tedlapu Hemavardhan Ram Sai, 324506402615, hemavardhantedlapu@gmail.com.

Tedlapu Hima Vardhan Laxman Sai, 324506402616, tedlapuhimavardhanlaxmansai@gmail.com.

Penugonda Manasa Lakshmi, Y24CS266, manasapenugonda07@gmail.com.

Revtish Muthineni, Y24CS294, revtishmuthineni@gmail.com.

ABSTRACT

This project presents the implementation of a static website hosting solution with a Continuous Integration and Continuous Deployment (CI/CD) pipeline using Amazon Web Services (AWS) to automate website deployment and improve delivery performance. The primary goal is to leverage modern DevOps practices to build an efficient, scalable, and event-driven solution that reduces manual deployment effort while ensuring continuous availability.

The system workflow begins with website source code being pushed to a GitHub repository. This event automatically triggers AWS CodePipeline, which retrieves the latest source files and deploys them to an Amazon S3 bucket configured for static website hosting. Amazon CloudFront, secured using Origin Access Control (OAC), distributes the website globally through edge locations, ensuring low latency access for users worldwide.

This architecture is fully cloud-native, utilizing S3 for storage, CloudFront for content delivery, CodePipeline for automated deployment, IAM for secure role-based permissions, CloudWatch for monitoring, and SNS for real-time email notifications. The solution offers high availability, reduced operational complexity, and faster response to website updates without any need for manual intervention or traditional server management.

By dynamically responding to source code changes and automating the deployment pipeline, the system ensures that the live website is always up to date and administrators are immediately informed of any error-rate anomalies through CloudWatch alarms. This enhances operational efficiency and supports reliable, production-ready web hosting. The project demonstrates how AWS cloud services and DevOps methodologies can be effectively applied to real-world deployment challenges, providing a robust foundation for future enhancements such as custom domains, infrastructure as code, and automated testing.

TABLE OF CONTENTS

1.....	INTRODUCTION
2.....	METHODOLOGY
3.....	TECHNOLOGY STACK
4.....	SYSTEM DESIGN / ARCHITECTURE
5.....	IMPLEMENTATION
6.....	RESULTS
7.....	CONCLUSION

1. INTRODUCTION

In the digital era, where agility, automation, and continuous delivery have become essential to business success, efficient website deployment has emerged as a critical operational need. This project, titled AWS Static Website Hosting with CI/CD, harnesses the power of cloud computing and DevOps automation on Amazon Web Services (AWS) to deliver a robust, scalable, and fully automated solution for hosting and deploying static websites. It stands as a testament to how cloud technologies can transform traditional manual deployment processes into intelligent, event-driven workflows with minimal infrastructure overhead.

Modern businesses demand uninterrupted website availability, fast global content delivery, and seamless deployment pipelines. Legacy hosting methods often fall short in scalability and flexibility, requiring manual file uploads and constant maintenance. This project meets those challenges by adopting a fully cloud-native architecture — where AWS services work in concert to automate source code integration, deployment, content delivery, and monitoring, all while maintaining high availability and operational efficiency.

Designed with simplicity, reliability, and automation at its core, the system allows developers to push website changes to a GitHub repository. This event automatically triggers AWS CodePipeline, which retrieves the latest source code and deploys it to an Amazon S3 bucket configured for static website hosting. Amazon CloudFront then distributes the updated content globally through edge locations, ensuring fast and secure access for end users.

What sets this project apart is its strategic use of AWS's managed services, eliminating the need for server provisioning, manual scaling, or ongoing infrastructure maintenance. Each component is triggered only when needed, enabling optimal resource usage and reduced costs. Additionally, the use of AWS IAM (Identity and Access Management) guarantees secure interactions between services, while Amazon CloudWatch and Amazon SNS facilitate system monitoring, alerting, and operational insights.

This report provides a comprehensive overview of the system's design, architecture, and implementation, highlighting the decision-making process, technical integrations, and benefits realized through this cloud-native, automated approach. The architecture promotes modularity and extensibility, enabling easy integration with future enhancements such as custom domain names, HTTPS certificates, Infrastructure as Code, and automated testing.

By delivering a real-time, event-driven, and fully automated static website hosting system, this project not only addresses current deployment needs but also showcases the immense potential of AWS DevOps services in solving real-world hosting challenges across industries.

2. METHODOLOGY

The development of the AWS Static Website Hosting with CI/CD system followed a structured and iterative methodology to ensure the successful fulfillment of project objectives. The methodology was divided into key phases, including requirements gathering, system design, implementation, testing, deployment, and refinement. Each phase was designed to align with best practices in cloud-native development and DevOps automation. The following subsections outline each phase in detail.

2.1. Requirements Gathering

The project began with a thorough analysis of the functional and technical requirements for an automated website hosting solution. Interactions with mentors and discussions around common deployment challenges helped define the core objectives: automate website deployment, improve global content delivery, and ensure secure, monitored infrastructure. The system was designed to detect GitHub commits, deploy updated files automatically, and alert administrators of any operational issues.

2.2. Design

Based on the identified requirements, a cloud-native, event-driven architecture was designed using AWS services. The workflow was defined to include GitHub as the source repository, AWS CodePipeline for deployment automation, Amazon S3 as the hosting layer, Amazon CloudFront for content delivery, and Amazon CloudWatch with SNS for monitoring and notifications. IAM roles and groups were planned to ensure secure interactions between services. Special attention was given to decoupling components and enabling scalability and fault tolerance.

2.3. Implementation

The implementation phase focused on translating the design into a fully functional cloud-hosted application. The GitHub repository was configured as the CodePipeline source stage, and the deploy stage was configured to publish files directly to the Amazon S3 bucket. Amazon CloudFront was configured with the S3 bucket as its origin and secured with Origin Access Control. The entire pipeline was validated using the AWS Management Console and the GitHub connection service.

2.4. Access & Security Management

A role-based approach was adopted for secure and scalable access control using AWS IAM. IAM groups were created for each functional area — S3, CloudFront, CloudWatch, CodePipeline, and Administration — and individual IAM users were assigned to the relevant groups. This structure supports fast permission management and allows seamless auditing of access across the project team.

2.5. Testing

Multiple levels of testing were carried out to validate functionality, deployment reliability, and system performance. Deployment tests validated that GitHub commits correctly triggered CodePipeline executions. Integration testing ensured that all AWS services — GitHub, CodePipeline, S3, CloudFront, IAM, CloudWatch, and SNS — worked together as intended. Functional testing simulated multiple website updates to confirm proper automatic deployment and content delivery.

2.6. Deployment

The system was deployed entirely on AWS using the Management Console. IAM roles and groups were carefully configured to grant least-privilege access between services. The CodePipeline was deployed with a GitHub connection to allow secure, authenticated access to the source repository. Amazon CloudWatch was used to monitor metrics, set up alarms, and ensure the reliability of the deployed infrastructure. The deployment process was optimized to enable real-time updates with minimal manual intervention.

2.7. Iterative Refinement

Throughout the development cycle, iterative feedback loops were established to continuously refine the system. Adjustments were made to IAM permissions, alarm thresholds, and pipeline configuration based on test results and mentor suggestions. The architecture was reviewed periodically to ensure adherence to AWS best practices. The iterative process ensured the system remained robust, maintainable, and ready for future enhancements such as custom domain integration or automated testing stages.

The methodology adopted for the development of this AWS static website hosting system ensured a cloud-native, scalable, and efficient solution. By following an event-driven and modular approach, the project successfully demonstrates the practical application of DevOps and AWS technologies in solving real-world website deployment challenges.

3. TECHNOLOGY STACK

The AWS Static Website Hosting with CI/CD system was developed using a modern, cloud-native technology stack that supports event-driven deployment, global content delivery, and continuous monitoring. The following technologies and services were used throughout the implementation:

3.1. Cloud Platform

- Amazon Web Services (AWS): The entire project is hosted on AWS, leveraging its managed services for storage, content delivery, deployment automation, and notifications.

3.2. AWS Services

- Amazon S3: Used for hosting the static website files. Serves as the deployment target and CloudFront origin.
- Amazon CloudFront: Content Delivery Network (CDN) that caches website content at global edge locations, secured using Origin Access Control (OAC).
- AWS CodePipeline: Automates the deployment workflow — detecting GitHub commits and deploying updated files to Amazon S3.
- AWS IAM (Identity and Access Management): Provides secure access control between AWS services by defining users, groups, roles, and permissions.
- Amazon CloudWatch: Monitors the system's performance through dashboards and alarms, tracking metrics such as requests, error rate, and bucket size.
- Amazon SNS (Simple Notification Service): Sends real-time email alerts whenever a CloudWatch alarm threshold is exceeded.

3.3. Version Control

- GitHub: Used as the source code repository, connected to AWS CodePipeline through a secure GitHub connection for automatic deployment triggers.

3.4. Web Technologies

- HTML5: Website structure, semantic elements, and content layout.
- CSS3: Styling, responsive design, and visual appearance.
- JavaScript: Interactive components and client-side functionality.

3.5. Security & Access

IAM Users, Groups, and Policies: Configured to allow only the necessary permissions for each service and team member, following the principle of least privilege. CloudFront Origin Access Control (OAC) further restricts direct access to the S3 origin.

3.6. Monitoring & Logging

- CloudWatch Dashboard: Visualizes real-time metrics such as Requests, BytesDownloaded, 5xxErrorRate, BucketSizeBytes, and NumberOfObjects.

- CloudWatch Alarms: Monitors thresholds (e.g., 5xx error rate) and triggers SNS notifications automatically.

4. SYSTEM DESIGN / ARCHITECTURE

The AWS Static Website Hosting with CI/CD system is designed as an event-driven, cloud-native solution that automates website deployment and global content delivery. The architecture adopts a fully managed AWS model, ensuring scalability, minimal infrastructure maintenance, and cost-efficiency. The system follows a multi-tier design, comprising the source control layer, deployment automation layer, hosting and content delivery layer, and monitoring layer, all powered by managed AWS services.

This section outlines the core components and their interactions in the architecture.

Figure 1 below presents the overall system architecture, showing how the developer, source control, deployment pipeline, hosting and content delivery layers, security, monitoring, and notification services interact end to end, together with the request/response workflow and the core technologies used.

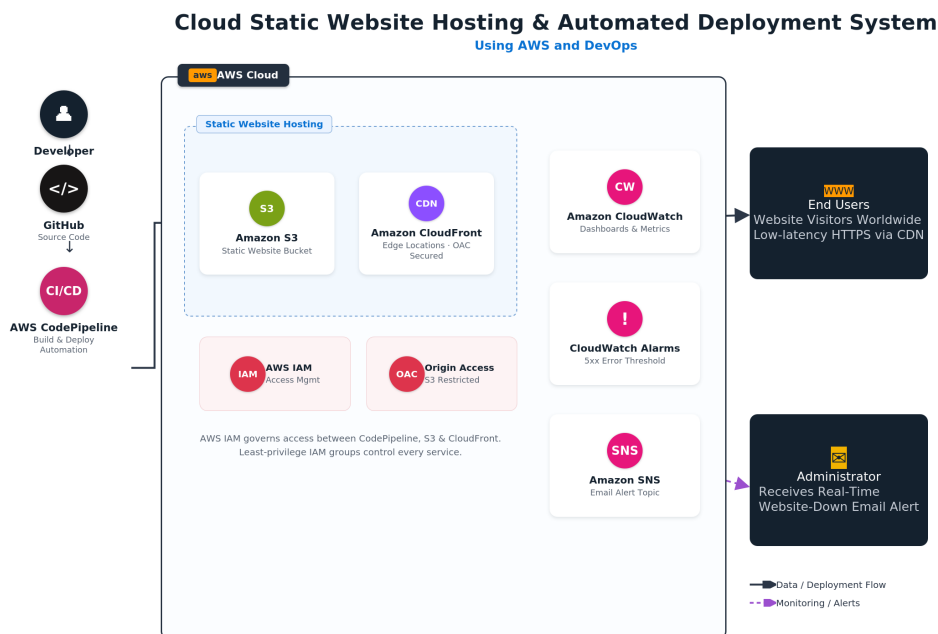


Figure 1. Overall System Architecture - AWS Static Website Hosting with CI/CD

Figure 1. Overall System Architecture – AWS Static Website Hosting with CI/CD

4.1. Source Control Layer – GitHub

The process begins when a developer pushes updated website files (HTML, CSS, JavaScript, images) to a GitHub repository. A secure GitHub connection, established through AWS Developer Tools, allows AWS CodePipeline to authenticate and access the repository. Any new commit to the tracked branch automatically triggers the pipeline.

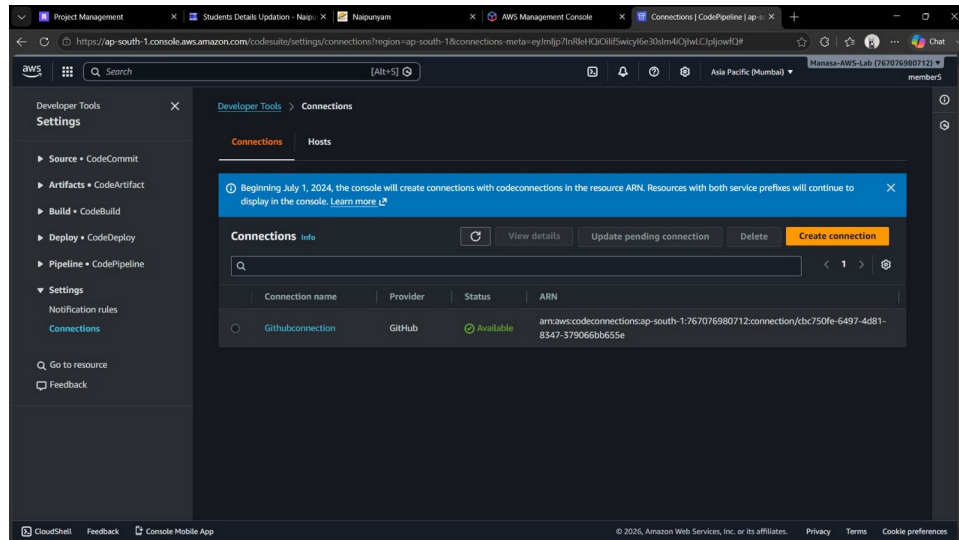


Figure 2. GitHub connection status in AWS Developer Tools

4.2. Deployment Automation Layer – AWS CodePipeline

An AWS CodePipeline is triggered by the GitHub source event. This pipeline performs the following tasks:

- Detects new commits pushed to the connected GitHub repository
- Retrieves the latest source files
- Deploys the updated files directly to the Amazon S3 bucket
- Confirms successful completion of both the Source and Deploy stages

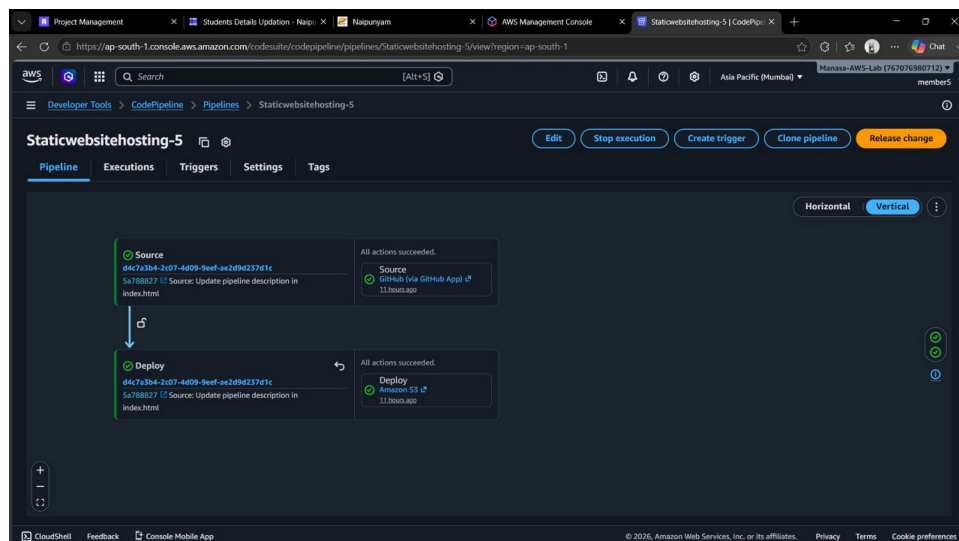


Figure 3. Staticwebsitehosting-5 pipeline with Source and Deploy stages succeeded

Execution ID	Status	Source	Trigger	Started	Duration	Completed
4b4963ac	Succeeded	Source - 459f327d: Update header text in index.html	StartPipelineExecution - member5	Jul 7, 2026 10:37 PM (UTC+5:30)	7 seconds	Jul 7, 2026 10:37 PM (UTC+5:30)
13d1ed54	Failed	Source - 459f327d: Update header text in index.html	StartPipelineExecution - member5	Jul 7, 2026 9:46 PM (UTC+5:30)	8 seconds	Jul 7, 2026 9:46 PM (UTC+5:30)
00c0aa3a	Succeeded	Source - 459f327d: Update header text in index.html	Commit 459f327d: pushed in revtishmuthinen/static-website-project/main Update header text in index.html	Jul 6, 2026 7:49 PM (UTC+5:30)	7 seconds	Jul 6, 2026 7:49 PM (UTC+5:30)
a193afbb	Succeeded	Source - 580ac065: Update index.html	Commit 580ac065: pushed in revtishmuthinen/static-website-project/main Update index.html	Jul 6, 2026 7:17 PM (UTC+5:30)	10 seconds	Jul 6, 2026 7:17 PM (UTC+5:30)
7f6d6236	Succeeded	Source - 689d5623: Add files via upload	Commit 689d5623: pushed in revtishmuthinen/static-website-project/main Add files via upload	Jul 6, 2026 7:15 PM (UTC+5:30)	8 seconds	Jul 6, 2026 7:15 PM (UTC+5:30)
a01521de	Succeeded	Source - 57d31af5: Delete logo.png.jpeg	Commit 57d31af5: pushed in revtishmuthinen/static-website-project/main Delete logo.png.jpeg	Jul 6, 2026 7:12 PM (UTC+5:30)	10 seconds	Jul 6, 2026 7:12 PM (UTC+5:30)

Figure 4. Pipeline execution history

Execution ID	Status	Source	Trigger	Started	Duration	Completed
ddc7a3b4	Succeeded	Source - 5a788827: Update pipeline description in index.html	Commit 5a788827: pushed in revtishmuthinen/static-website-project/main Update pipeline description in index.html	Jul 8, 2026 9:32 AM (UTC+5:30)	6 seconds	Jul 8, 2026 9:32 AM (UTC+5:30)
3ab7f8b6	Succeeded	Source - 760b8aac: Update header and add codepipeline paragraph	Commit 760b8aac: pushed in revtishmuthinen/static-website-project/main Update header and add codepipeline paragraph	Jul 7, 2026 11:07 PM (UTC+5:30)	6 seconds	Jul 7, 2026 11:07 PM (UTC+5:30)
fd59d619	Succeeded	Source - d2a722d6: Add heading for codepipeline section	Commit d2a722d6: pushed in revtishmuthinen/static-website-project/main Add heading for codepipeline section	Jul 7, 2026 10:55 PM (UTC+5:30)	8 seconds	Jul 7, 2026 10:55 PM (UTC+5:30)
8cd6fd5	Succeeded	Source - 713186c8: Update title in index.html	Commit 713186c8: pushed in revtishmuthinen/static-website-project/main Update title in index.html	Jul 7, 2026 10:45 PM (UTC+5:30)	9 seconds	Jul 7, 2026 10:45 PM (UTC+5:30)

Figure 5. Additional successful pipeline executions

4.3. Hosting Layer – Amazon S3

Website files are hosted in an Amazon S3 bucket configured for static website hosting. This bucket receives updated files automatically from CodePipeline and serves as the origin for the CloudFront distribution.

4.4. Content Delivery Layer – Amazon CloudFront

Amazon CloudFront distributes the website content globally through edge locations, providing low-latency access for users. The distribution is secured using Origin Access Control (OAC), which ensures that the S3 origin can only be accessed through CloudFront, not directly by the public.

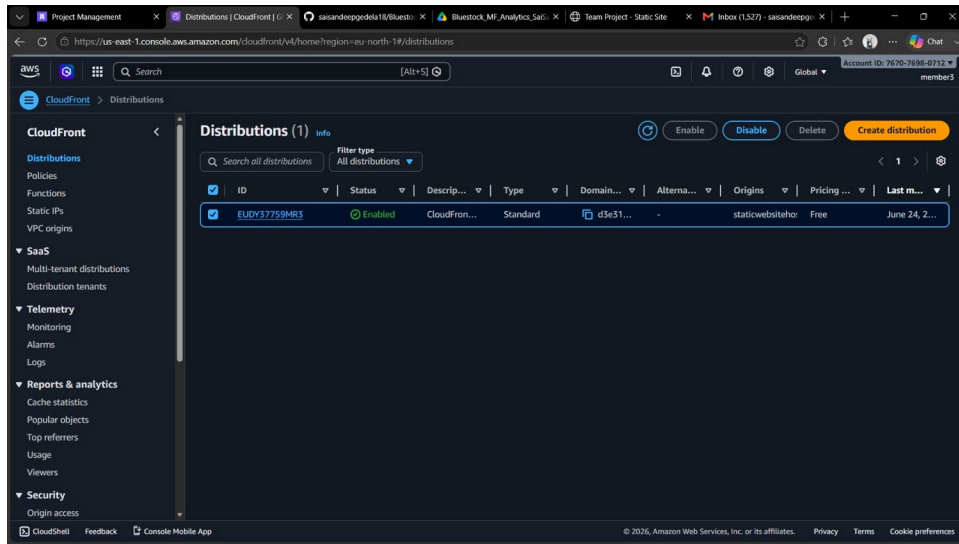


Figure 6. CloudFront distribution list – status Enabled

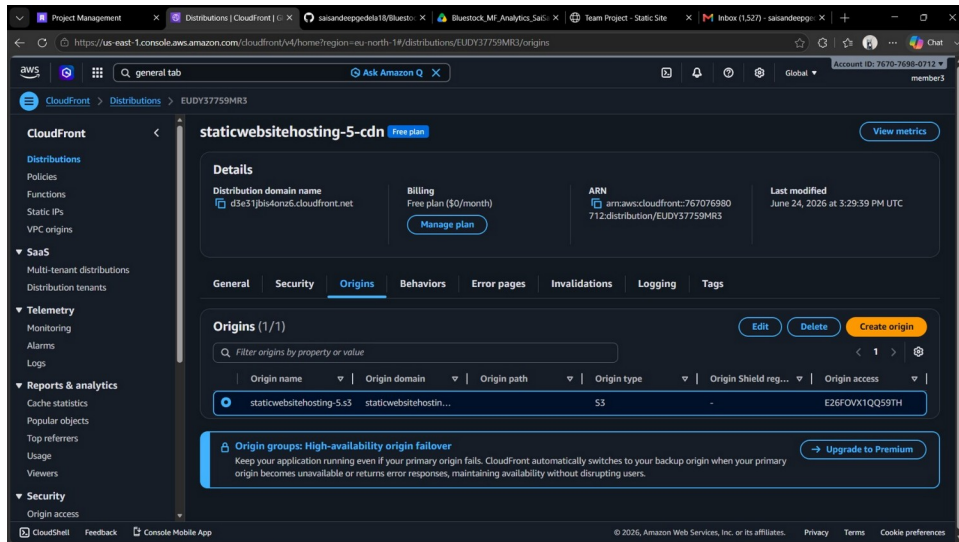


Figure 7. CloudFront origin configured with Origin Access Control

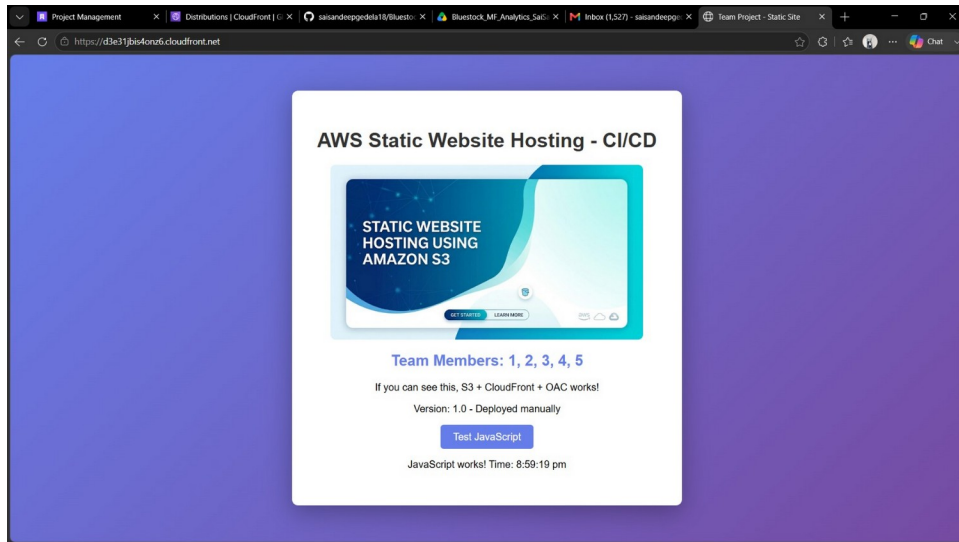


Figure 8. Live website served through the CloudFront domain

4.5. Security & Access Control – AWS IAM

AWS Identity and Access Management (IAM) is used to securely grant permissions between team members and services:

- Each team member is assigned to an IAM group corresponding to their responsibility (S3, CloudFront, CloudWatch, CodePipeline, or Administration)
- Policies are attached at the group level to enforce consistent, least-privilege access
- Individual inline or customer-managed policies are added only where required

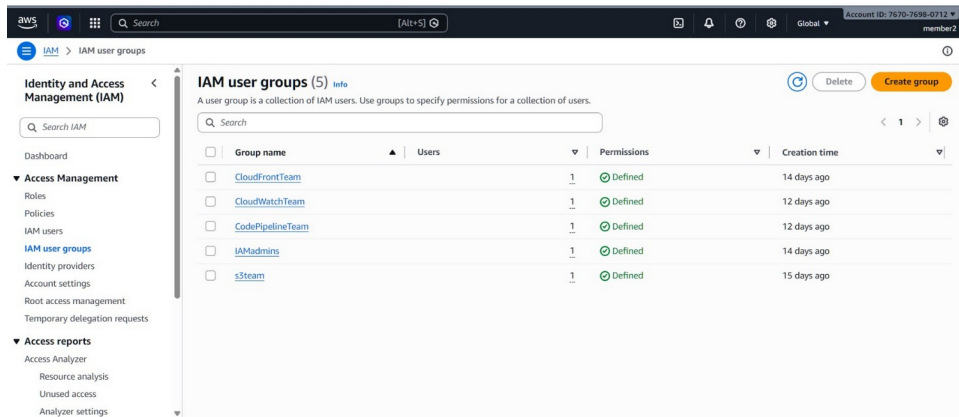


Figure 9. IAM user groups – CloudFrontTeam, CloudWatchTeam, CodePipelineTeam, IAMadmins, s3team

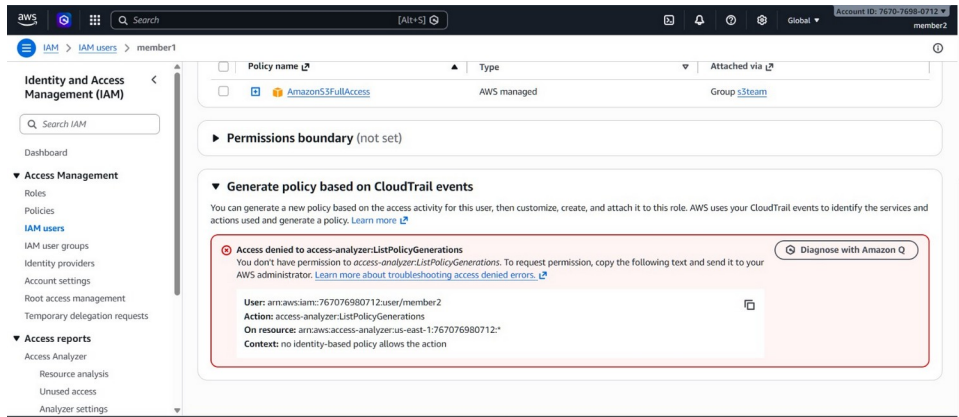


Figure 10. member1 permissions – AmazonS3FullAccess via s3team

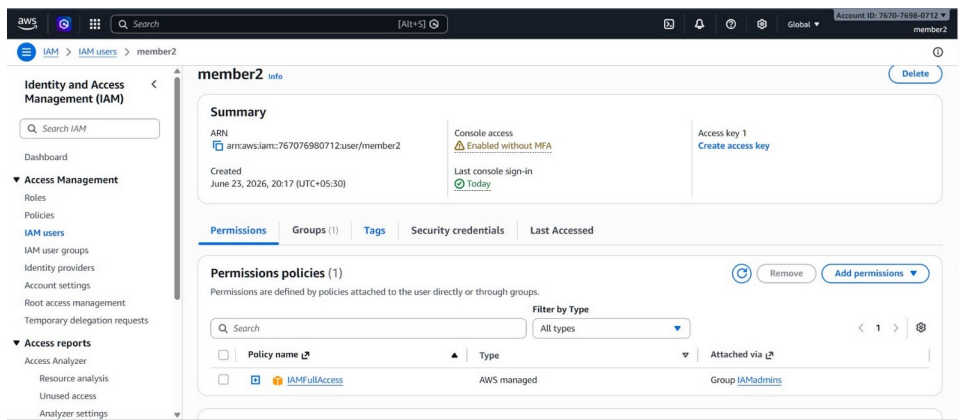


Figure 11. member2 permissions – IAMFullAccess via IAMadmins

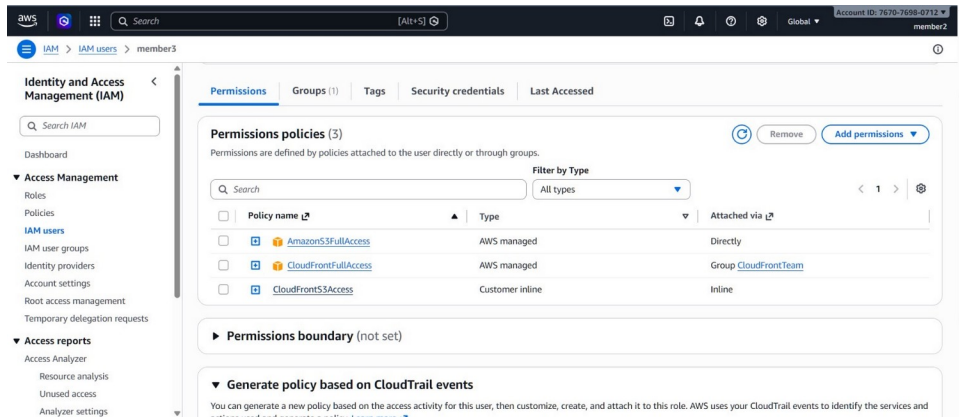


Figure 12. member3 permissions – S3 and CloudFront access

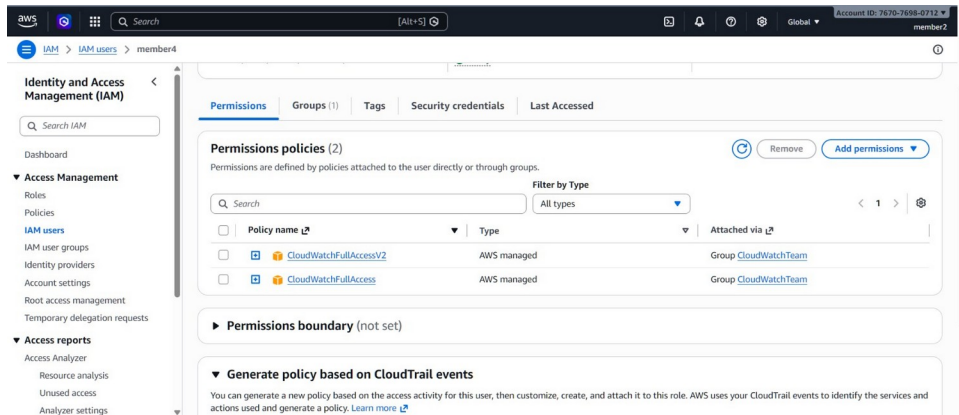


Figure 13. member4 permissions – CloudWatchFullAccess via CloudWatchTeam

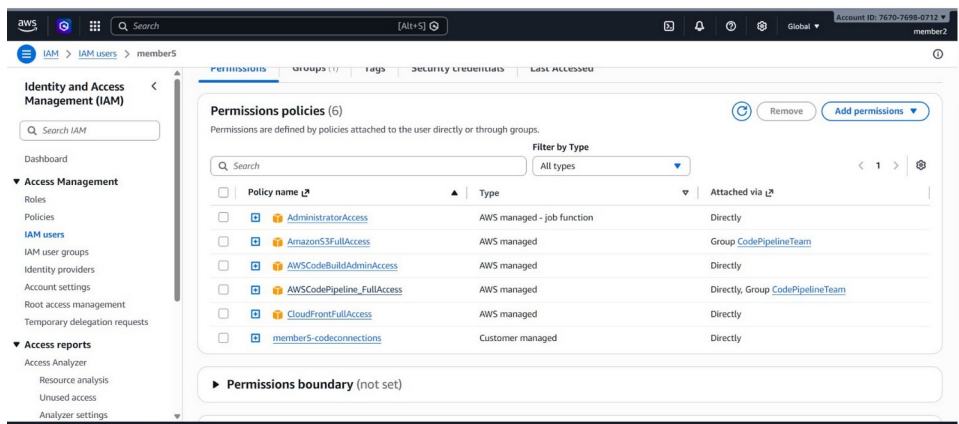


Figure 14. member5 permissions – Administrator and CodePipeline access

4.6. Monitoring and Logging – Amazon CloudWatch

Amazon CloudWatch is integrated for:

- Real-time monitoring of website requests, error rate, and bucket metrics
- Visualizing metrics on a centralized dashboard
- Setting alarms for abnormal 5xx error activity

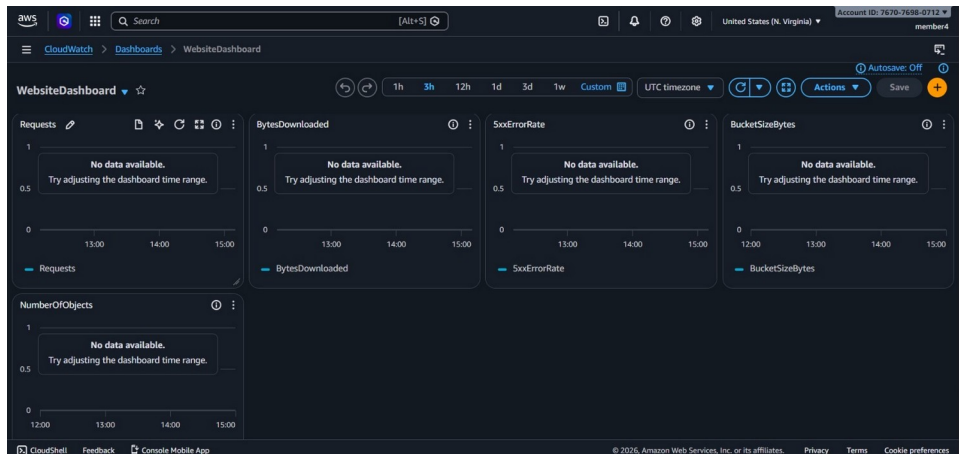


Figure 15. WebsiteDashboard – Requests, BytesDownloaded, 5xxErrorRate, BucketSizeBytes, NumberOfObjects

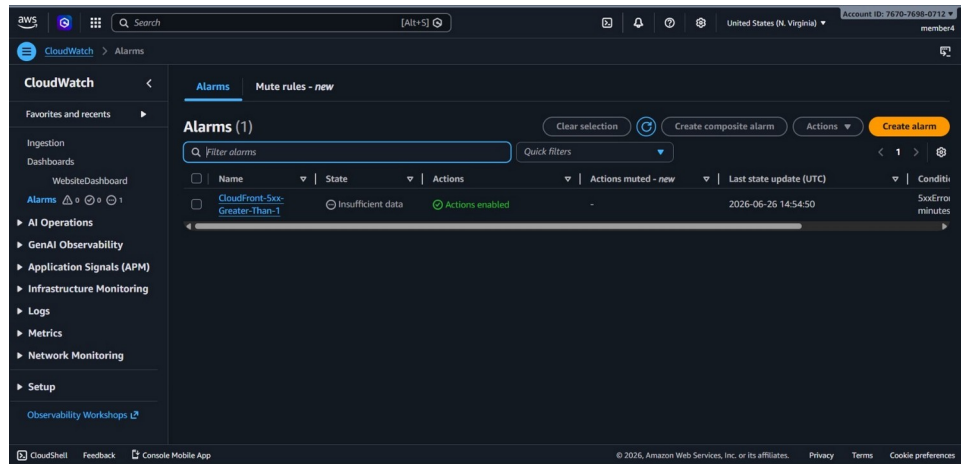


Figure 16. CloudWatch Alarms list

Monitoring was subsequently extended to a second S3 bucket (pipeline4) with dedicated CloudWatch alarms tracking bucket size and object count. Both alarms — S3-BucketSize-pipeline4 and S3-NumberOfObjects-pipeline4 — are shown below in an In alarm state, confirming that threshold-based monitoring is active for this additional bucket as well.

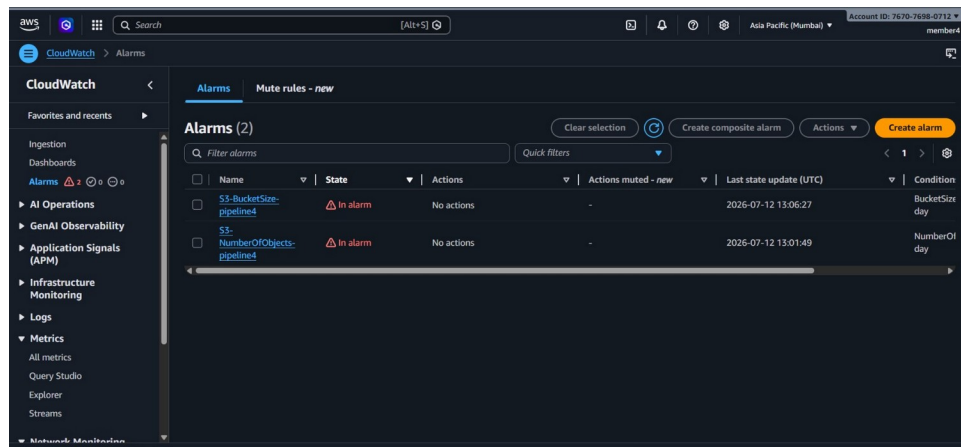


Figure 17. CloudWatch Alarms for the second S3 bucket (pipeline4) – S3-BucketSize-pipeline4 and S3-NumberOfObjects-pipeline4

4.7. Notification Layer – Amazon SNS

Once a CloudWatch alarm enters the ALARM state, it publishes a message to an Amazon SNS topic, which is subscribed to by the administrator via email. SNS ensures real-time delivery of alerts, allowing the team to take immediate action.

8:30

0.30 KB/s 52



Subscription confirm



Simple Notification Service

Subscription confirmed!

You have successfully subscribed.

Your subscription's id is:

arn:aws:sns:us-east-1:767076980712:Website-Down:0966194b-1438-4e52-b7d1-eaac5add126d

If it was not your intention to subscribe, [click here to unsubscribe](#).

Figure 18. SNS email subscription confirmed

5. IMPLEMENTATION

The implementation of the AWS Static Website Hosting with CI/CD system translates the architectural design into a fully functional cloud-based solution. This section outlines how each AWS service and component was configured and developed to work together in an automated, event-driven workflow. The implementation focused on enabling real-time website updates, automated deployment, and alert mechanisms — all without managing any physical servers.

5.1. Website Development

The project began with a responsive static website consisting of a home page, navigation bar, responsive layout, CSS styling, JavaScript functionality, and images/static assets (index.html, style.css, script.js, banner image). These files serve as the deployment artifacts throughout the project.

5.2. GitHub Repository & Connection Setup

A GitHub repository was created to maintain the website source code. A connection was authorized between AWS Developer Tools and GitHub (Githubconnection) so that AWS CodePipeline could securely access the repository and detect new commits.

5.3. Amazon S3 Bucket Configuration

Amazon S3 was configured as the primary storage service for hosting the static website. A new S3 bucket was created, and the bucket policy was configured to allow access only via CloudFront, keeping the origin storage secure.

5.4. Amazon CloudFront Configuration

A CloudFront distribution was created with the S3 bucket as origin, the default root object set to index.html, HTTPS enabled, and caching configured. After deployment completed, the website became globally accessible through the CloudFront domain.

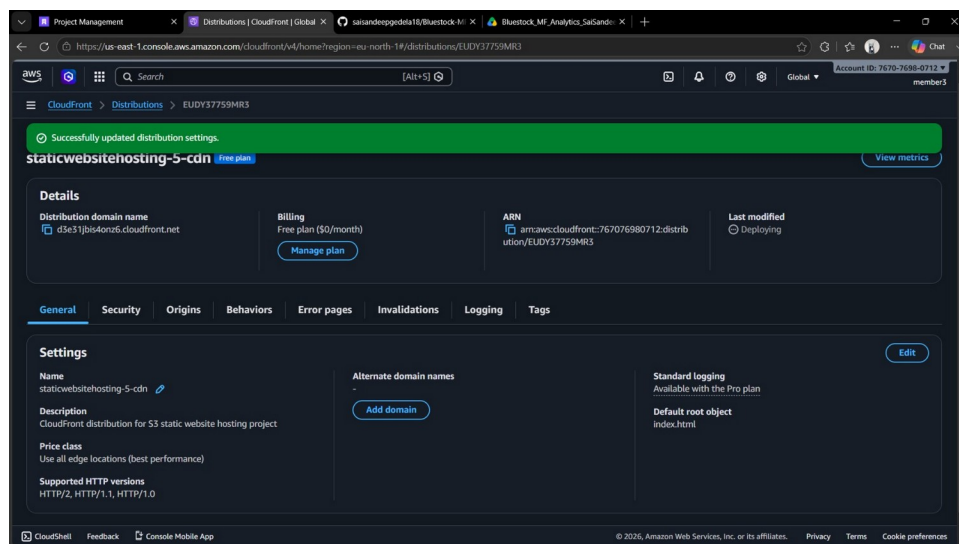


Figure 19. CloudFront distribution settings updated – deployment in progress

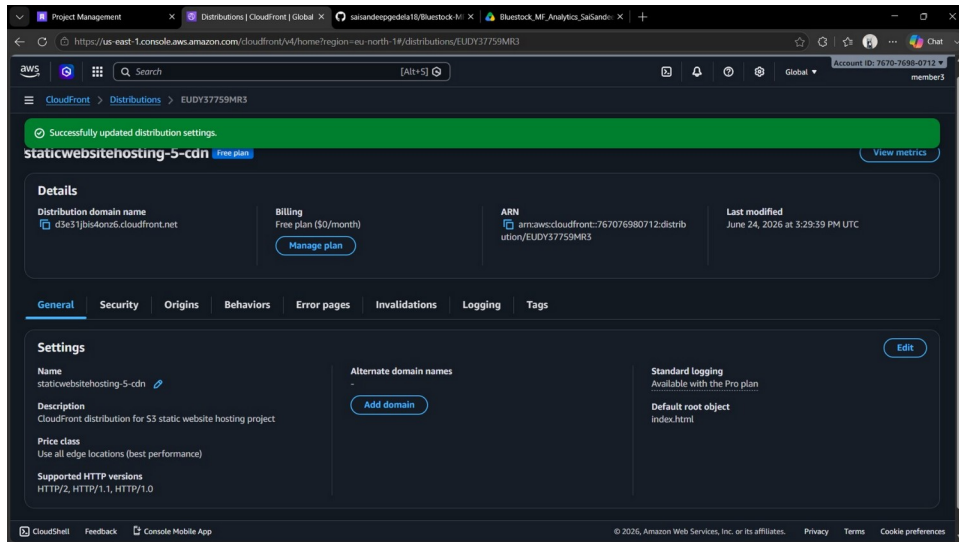


Figure 20. CloudFront distribution fully deployed with domain `d3e31jbis4onz6.cloudfront.net`

5.5. Origin Access Control (OAC)

To enhance security, Origin Access Control (OAC) was created and attached to the CloudFront distribution, and the Amazon S3 bucket policy was updated to restrict direct public access, allowing access only through CloudFront. This prevents direct S3 access, improves security, and follows AWS recommended best practices.

5.6. IAM Users and Groups

AWS IAM was implemented to provide secure access control. Five IAM groups were created — `s3team`, `IAMadmins`, `CloudFrontTeam`, `CloudWatchTeam`, and `CodePipelineTeam` — and individual IAM users were added to the relevant groups, following the Principle of Least Privilege while assigning permissions.

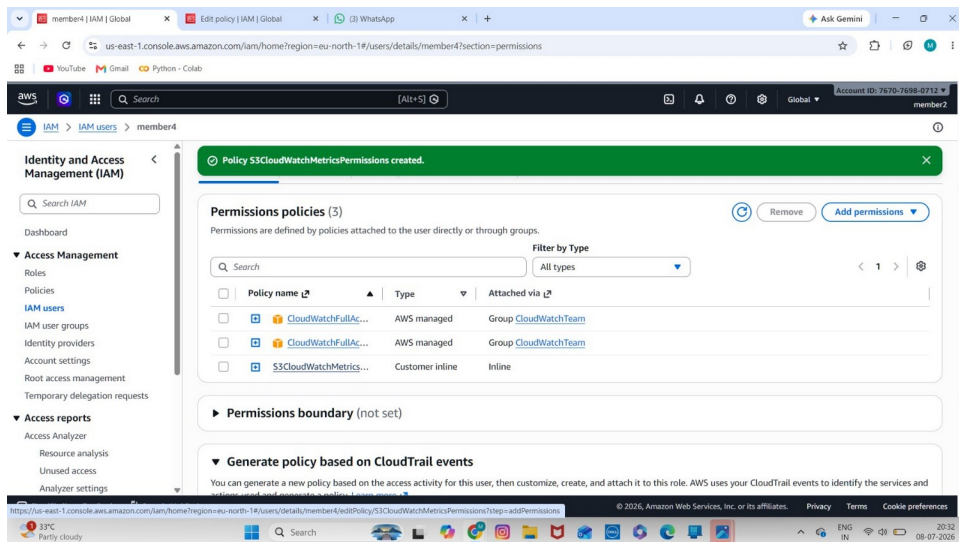


Figure 21. Custom inline policy `S3CloudWatchMetricsPermissions` attached to `member4`

5.7. AWS CodePipeline Configuration

AWS CodePipeline was configured with a Source stage connected to the GitHub repository and a Deploy stage targeting the Amazon S3 bucket. Whenever changes are pushed to GitHub, the pipeline automatically deploys the updated website to Amazon S3, eliminating manual deployment and ensuring continuous delivery.

5.8. CloudWatch Dashboard Implementation

An Amazon CloudWatch dashboard named WebsiteDashboard was created to monitor CloudFront and S3 metrics, including Requests, BytesDownloaded, 5xxErrorRate, BucketSizeBytes, and NumberOfObjects.

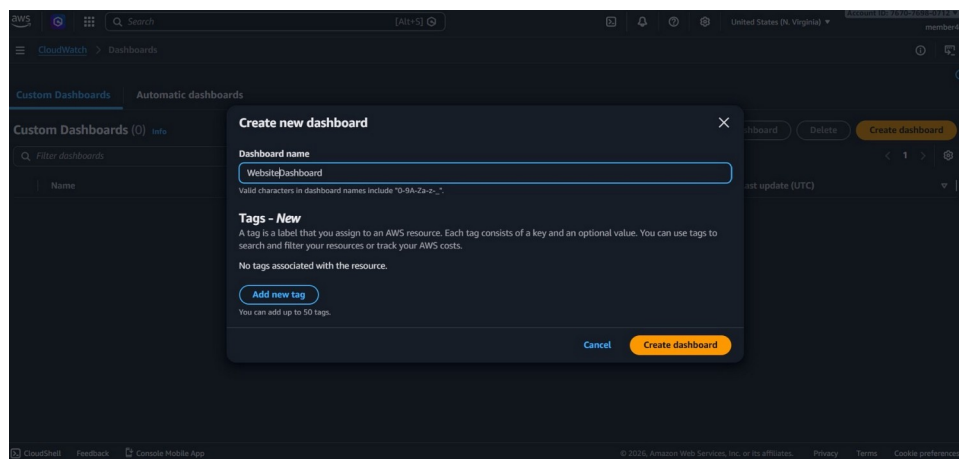


Figure 22. Creating a new CloudWatch dashboard named WebsiteDashboard

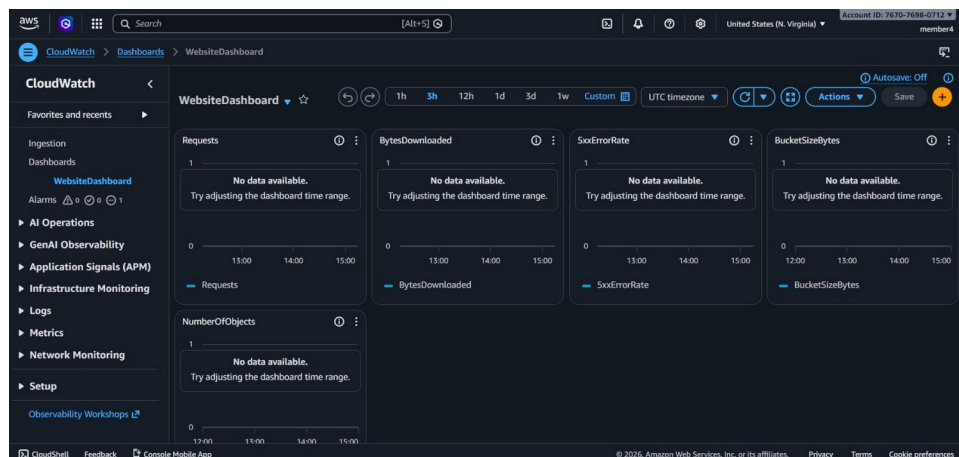


Figure 23. WebsiteDashboard – full widget layout

5.9. CloudWatch Alarm Configuration

A CloudWatch alarm named CloudFront-5xx-Greater-Than-1 was created on the CloudFront 5xxErrorRate metric. A notification action was configured to publish to the "Website-Down" SNS topic whenever the alarm enters the ALARM state.

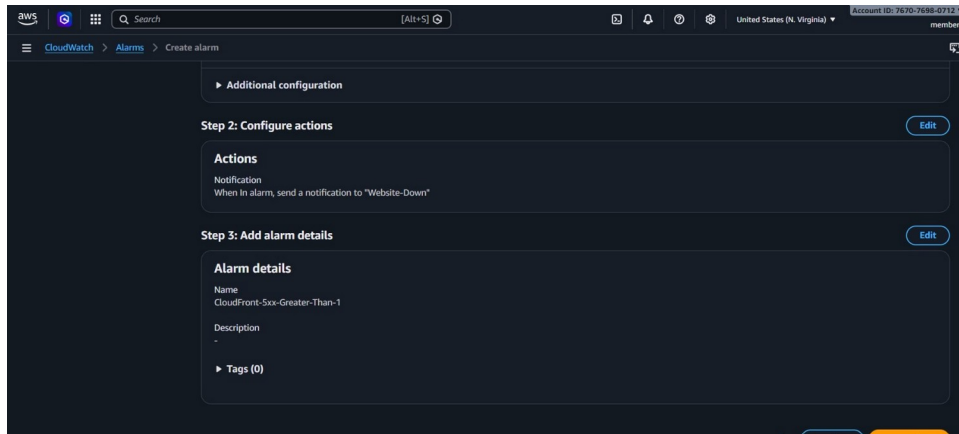


Figure 24. Alarm configuration – notification action linked to the Website-Down topic

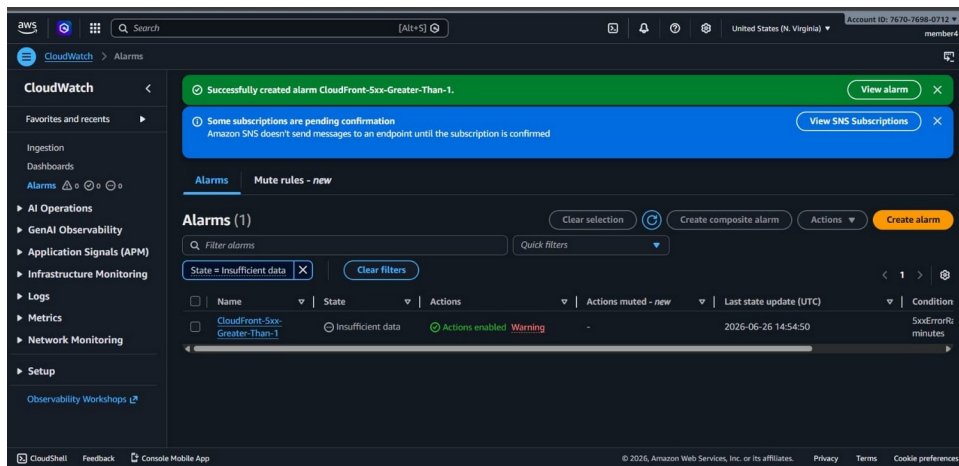


Figure 25. Alarm created, with an SNS subscription pending confirmation

Alarm coverage was later extended to a second S3 bucket (pipeline4), with alarms configured on bucket size and object count metrics — see Figure 17 in Section 4.6.

5.10. Amazon SNS Integration

An SNS topic was created and an email subscription was added. After confirming the subscription, the CloudWatch alarm was linked to the SNS topic so that administrators receive immediate email notifications whenever the alarm threshold is exceeded.

5.11. Testing and Validation

- Website Testing: Website loaded successfully; responsive layout verified; images displayed correctly; navigation links validated.
- CloudFront Testing: Website accessible through the CloudFront URL; HTTPS working successfully; caching verified.
- CodePipeline Testing: GitHub push detected successfully; pipeline executed automatically; deployment completed successfully.
- IAM Testing: Permissions verified; unauthorized access prevented; role-based access confirmed.

- Monitoring Testing: Dashboard displayed metrics correctly; alarm created successfully; SNS notification received.

The successful implementation of this system showcases how AWS services can be integrated to create a real-time, low-maintenance, and scalable solution for static website hosting. The modular and event-driven approach ensures flexibility for future enhancements such as custom domains, automated testing, or Infrastructure as Code.

6. RESULTS

The implementation of the AWS Static Website Hosting with CI/CD system yielded several successful outcomes that align with the project's objectives. The system was tested under multiple scenarios to validate functionality, performance, scalability, and reliability. The following results highlight the effectiveness of using an automated AWS DevOps pipeline for real-time static website deployment.

6.1. Automated Website Deployment

Whenever changes were pushed to the GitHub repository, AWS CodePipeline automatically detected the commit, retrieved the updated source files, and deployed them to Amazon S3. The updated website became available through CloudFront within seconds, confirming that the CI/CD mechanism is reliable and timely.

6.2. Global Content Delivery

Amazon CloudFront successfully distributed website content through edge locations worldwide. The website was accessible through the CloudFront domain (d3e31jbis4onz6.cloudfront.net) with faster page loading through caching, and the S3 origin remained inaccessible directly, confirming the Origin Access Control configuration was effective.

6.3. Scalable and Automated Architecture

The system handled multiple deployments in quick succession without any manual scaling or downtime, thanks to the underlying managed AWS infrastructure. The combination of GitHub, CodePipeline, S3, and CloudFront ensures that each component functions independently and scales automatically based on demand.

Testing scenarios included:

- Pushing multiple commits with website content changes
- Verifying pipeline execution history for consistent success
- Monitoring CloudWatch metrics and dashboard behaviour during deployments

All scenarios were handled successfully without performance degradation.

6.4. Secure and Controlled Service Access

Using AWS IAM users, groups, and policies, secure access was configured between all services:

- IAM users could only access resources required for their assigned responsibility
- The S3 origin was restricted to CloudFront access only, via OAC
- SNS topic subscriptions were managed through a verified email endpoint

This validated that the system follows best practices for cloud security and permissions.

6.5. System Monitoring and Logging

Through Amazon CloudWatch, the system was monitored for request counts, error rates, bucket size, and object counts. The CloudFront-5xx-Greater-Than-1 alarm was successfully created and linked to the Website-Down SNS topic, confirming that the monitoring and alerting pipeline functions as intended. Monitoring was further extended to a second S3 bucket (pipeline4), where the S3-BucketSize-pipeline4 and

S3-NumberOfObjects-pipeline4 alarms were observed in an In alarm state, demonstrating that the alerting mechanism correctly flags threshold breaches across multiple buckets.

6.6. User Experience and Functionality

The system requires minimal manual interaction:

- Pushing code to GitHub is the only manual step
- All subsequent actions – build detection, deployment, and cache delivery – are fully automated
- Alerts are sent in a clear, human-readable email format

This ease of use confirms the system's potential for real-world deployment in production hosting environments.

The overall results demonstrate that the AWS static website hosting system meets its intended goals of automation, efficiency, scalability, and ease of use. The architecture is stable and ready for real-time operational use, with opportunities for further expansion and integration.

6.7. Screenshots of the Output

Representative screenshots of the live website, S3/CloudFront configuration, IAM users and groups, GitHub connection, CodePipeline executions, CloudWatch dashboard and alarms (including the second-bucket alarms), and the SNS subscription are included in Chapter 4 (System Design / Architecture) and Chapter 5 (Implementation) as Figures 1–25.

7. CONCLUSION

The successful development and deployment of the AWS Static Website Hosting with CI/CD system demonstrate the power and practicality of cloud-native, event-driven architectures in solving real-world deployment challenges. By leveraging key AWS services such as S3, CloudFront, CodePipeline, IAM, CloudWatch, and SNS, the system achieves its core objectives: automating website deployment, securing content delivery, and delivering real-time monitoring alerts — all without managing any physical infrastructure.

The project highlights the benefits of AWS DevOps automation, including automatic scaling, cost-efficiency, fault tolerance, and reduced operational complexity. The use of AWS CodePipeline allows for on-demand deployment of website updates, while Amazon CloudFront provides fast and reliable global content delivery. The SNS-based notification mechanism ensures that administrators are alerted immediately when error rates rise, enabling timely troubleshooting and response.

Throughout the development lifecycle, careful attention was given to security, performance, and modularity. The system was designed using best practices for IAM permissions, Origin Access Control, and monitoring via CloudWatch. Testing confirmed the system's ability to process various deployment scenarios reliably.

Beyond its current functionality, the project lays a strong foundation for future enhancements, such as:

- Integrating a custom domain using Amazon Route 53
- Enabling HTTPS certificates through AWS Certificate Manager (ACM)
- Adopting Infrastructure as Code (IaC) using Terraform or AWS CloudFormation
- Adding automated testing stages within the CI/CD pipeline
- Extending monitoring with CloudWatch Logs Insights and AWS WAF for enhanced security

In conclusion, this project serves as a practical and scalable blueprint for AWS-based DevOps application development, particularly in domains where automation and continuous delivery are critical. It showcases how cloud technologies can be strategically combined to build efficient, secure, and future-ready web hosting solutions with minimal operational burden.